

SYSTEM ANALYSIS & ARCHITECTURE DIAGRAMS

A Comprehensive, Precision-Engineered Reference Guide to 12 Core Visual Modeling Standards

Diagram Tool	Core Focus & Purpose	Key Modeling Standards
BPMN 2.0	Business Process Flow & Accountability	ISO/IEC 19510 Standard
Use Case Diagram	Actor & System Goal Interactions	UML 2.5 Behavior Modeling
ER Diagram	Database Entity Relationships & Constraints	Crow's Foot Relational Schema
Data Flow Diagram	Information Transformation & Storage Flow	Gane & Sarson DFD Level 1
Wireframe / Mockup	Low-Fidelity UI Layout & Information Hierarchy	UX/UI Skeletal Specification
Swimlane Diagram	Cross-Functional Departmental Accountability	Functional Matrix Flowchart
Gantt Chart	Project Timeline, Tasks & Dependencies	WBS Project Scheduling
Stakeholder Map	Power vs. Interest Quadrant Matrix	Mendelow Power-Interest Grid
Statechart Diagram	Object Lifecycle State Transitions	UML 2.5 State Machine
RTM Matrix	Requirement-to-Test & Defect Traceability	IEEE 830 / ISO 29148 Standard
Gap Analysis	Current vs. Future State Transition Path	Strategic Capability Assessment
Activity Diagram	Algorithmic Logic & Concurrent Workflows	UML 2.5 Workflow Control

1. Business Process Model and Notation (BPMN 2.0)

Core Architectural Purpose: BPMN is a standardized visual language designed to map out complex business workflows with explicit precision. Unlike basic flowcharts, BPMN differentiates between events (triggers), activities (work performed), decision gateways (branching/merging), and organizational responsibilities via pools and swimlanes.

Detailed Scenario Walkthrough: Monitoring hot food display temperatures in a service station and executing the shift closing procedure, explicitly highlighting the temperature drop bottleneck.

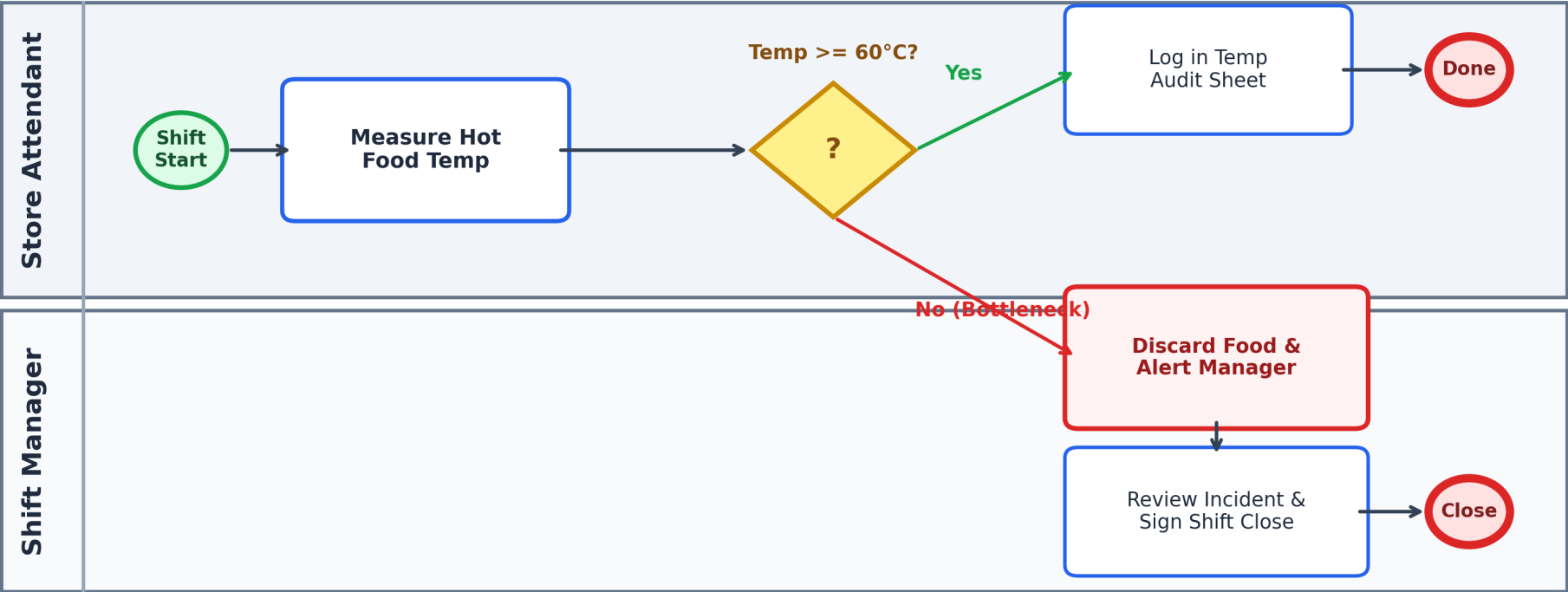
Standard Notations & Visual Grammar Matrix:

Symbol / Notation	Visual Feature	Architectural Semantics
Event (Circle)	Triggers or marks status shifts in a process.	Start Event (thin), Intermediate Event (double), End Event (thick).
Activity (Rounded Rect)	Represents actionable work requiring human or system effort.	Must contain active verb (e.g., 'Measure Food Temp').
Gateway (Diamond)	Controls branching and merging of sequence flows based on logic.	Exclusive Gateway (X) enforces mutually exclusive paths.
Pool & Swimlane	Establishes organizational accountability across distinct roles.	Lane 1: Store Attendant, Lane 2: Shift Manager.

Key Design Insight: Ensure every sequence flow arrow connects seamlessly to node centers without intersecting borders. Explicitly label decision branches (Yes/No) and highlight bottleneck nodes in warning colors.

1. Business Process Model and Notation (BPMN 2.0) — High-Res Precision Visual Model

BPMN 2.0: Hot Food Temp Monitoring & Shift Close Flow



Key Design Insight: Diagram Model Execution Note: This model illustrates the 'Monitoring hot food display temperatures in a service station and executing the shift closing procedure, explicitly highlighting the temperature drop bottleneck.' scenario. All shapes, line anchors, and decision labels are rendered with precise spatial alignment.

2. Use Case Diagram (UML 2.5)

Core Architectural Purpose: A Use Case diagram captures the functional requirements of a system by portraying the relationships between actors (users/systems) and key use cases (system capabilities) within a defined boundary.

Detailed Scenario Walkthrough: Modeling a Student actor interacting with a career platform to book events, take AI mock interviews, and upload resumes, with an automated feedback inclusion.

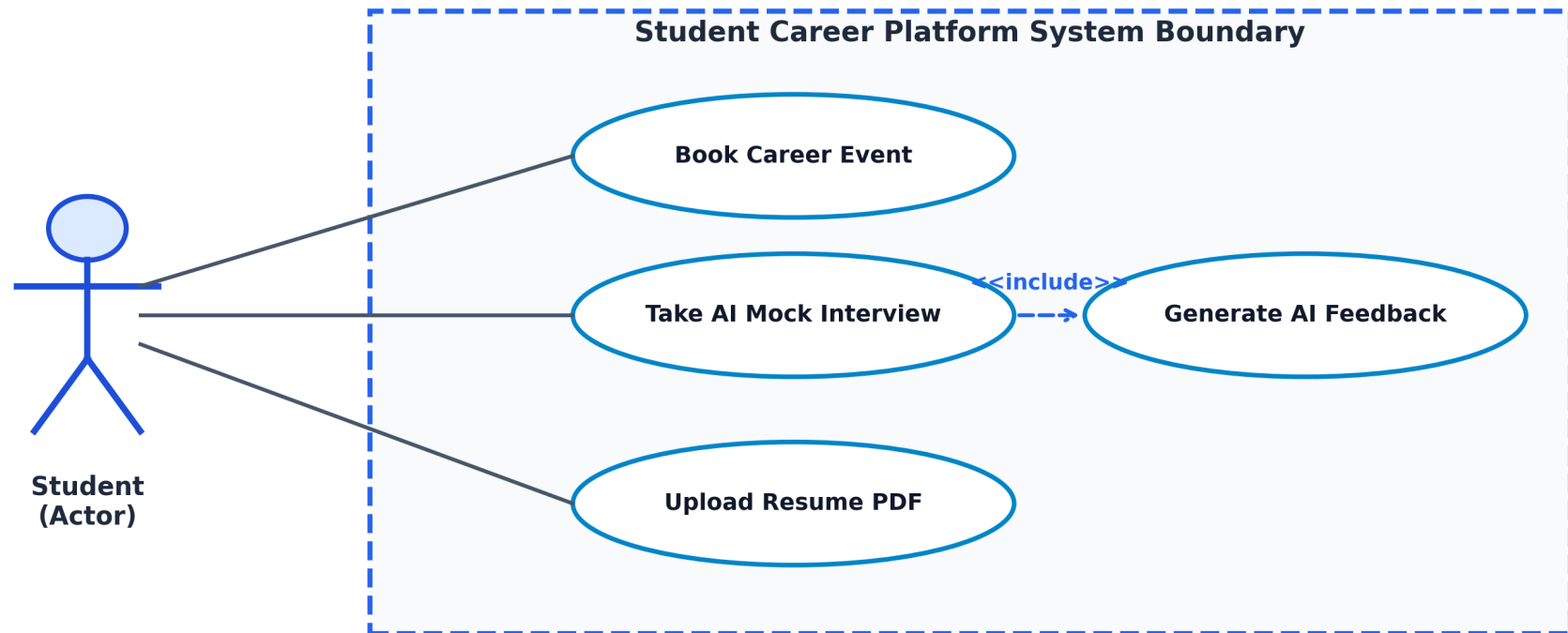
Standard Notations & Visual Grammar Matrix:

Symbol / Notation	Visual Feature	Architectural Semantics
Actor (Stick Figure)	Represents a role played by an external entity interacting with system.	Placed outside the system boundary box.
Use Case (Oval)	Represents a discrete unit of system functionality.	Enclosed inside the System Boundary box.
System Boundary (Rect)	Defines the scope and limits of the software system.	Rendered as a dashed border box with system title.
<> Relation	Indicates a compulsory sub-routine triggered by a base use case.	Dashed arrow pointing from base to included use case.

Key Design Insight: Maintain clear spacing between actor stick figures and use case ovals so association lines never cross over text labels.

2. Use Case Diagram (UML 2.5) — High-Res Precision Visual Model

UML Use Case Diagram: Student Actor & System Interactions



Key Design Insight: Diagram Model Execution Note: This model illustrates the 'Modeling a Student actor interacting with a career platform to book events, take AI mock interviews, and upload resumes, with an automated feedback inclusion.' scenario. All shapes, line anchors, and decision labels are rendered with precise spatial alignment.

3. Entity-Relationship (ER) Diagram

Core Architectural Purpose: An ER Diagram visualizes the conceptual and logical data architecture of a database, illustrating entities, attributes, and relationship constraints.

Detailed Scenario Walkthrough: Modeling a 1-to-Many relational structure between a Student Profile entity and multiple Job Application records.

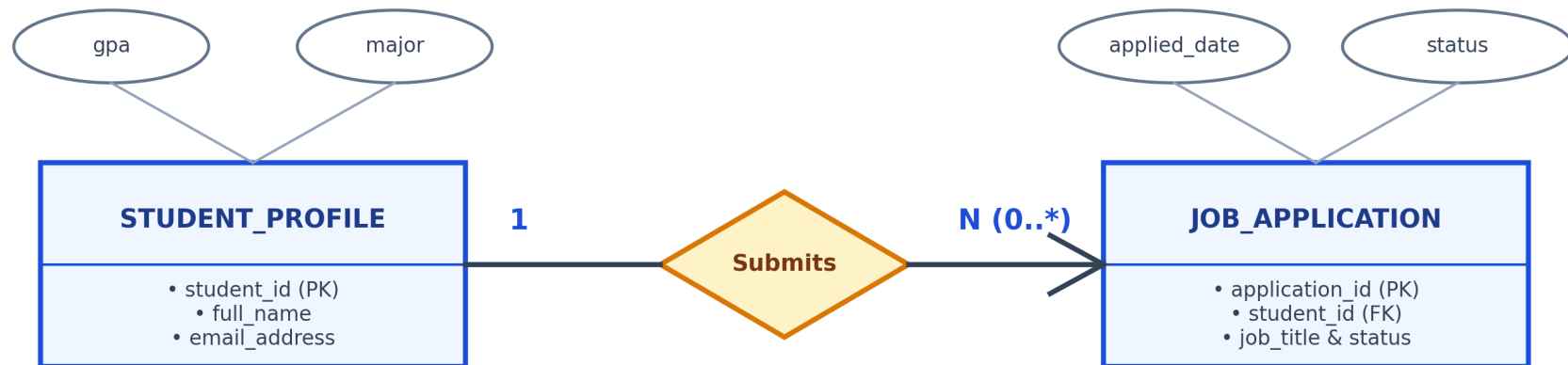
Standard Notations & Visual Grammar Matrix:

Symbol / Notation	Visual Feature	Architectural Semantics
Entity (Rectangle)	A real-world object or dataset stored in database.	STUDENT_PROFILE and JOB_APPLICATION entities.
Attribute (Oval)	Specific data fields or properties describing an entity.	student_id (PK), email_address, application_id.
Relationship (Diamond)	Defines how two entities relate to each other.	Submits / Places relationship diamond.
Crow's Foot Notation	Expresses relational cardinality and optionality constraints.	1-side uses hash mark; N-side uses 3-pronged branch.

Key Design Insight: Align primary key (PK) and foreign key (FK) attributes at the top of entity boxes, and position cardinality markers cleanly near border edges.

3. Entity-Relationship (ER) Diagram — High-Res Precision Visual Model

Entity-Relationship (ER) Diagram: 1-to-Many Student Application Model



Key Design Insight: Diagram Model Execution Note: This model illustrates the 'Modeling a 1-to-Many relational structure between a Student Profile entity and multiple Job Application records.' scenario. All shapes, line anchors, and decision labels are rendered with precise spatial alignment.

4. Data Flow Diagram (DFD Level 1)

Core Architectural Purpose: A DFD maps the transformation and movement of data within a system, detailing where information originates, how it is processed, where it is stored, and where it exits.

Detailed Scenario Walkthrough: Tracing a user's resume PDF file from mobile upload, through the AI parsing engine and vector matcher, into the secure database.

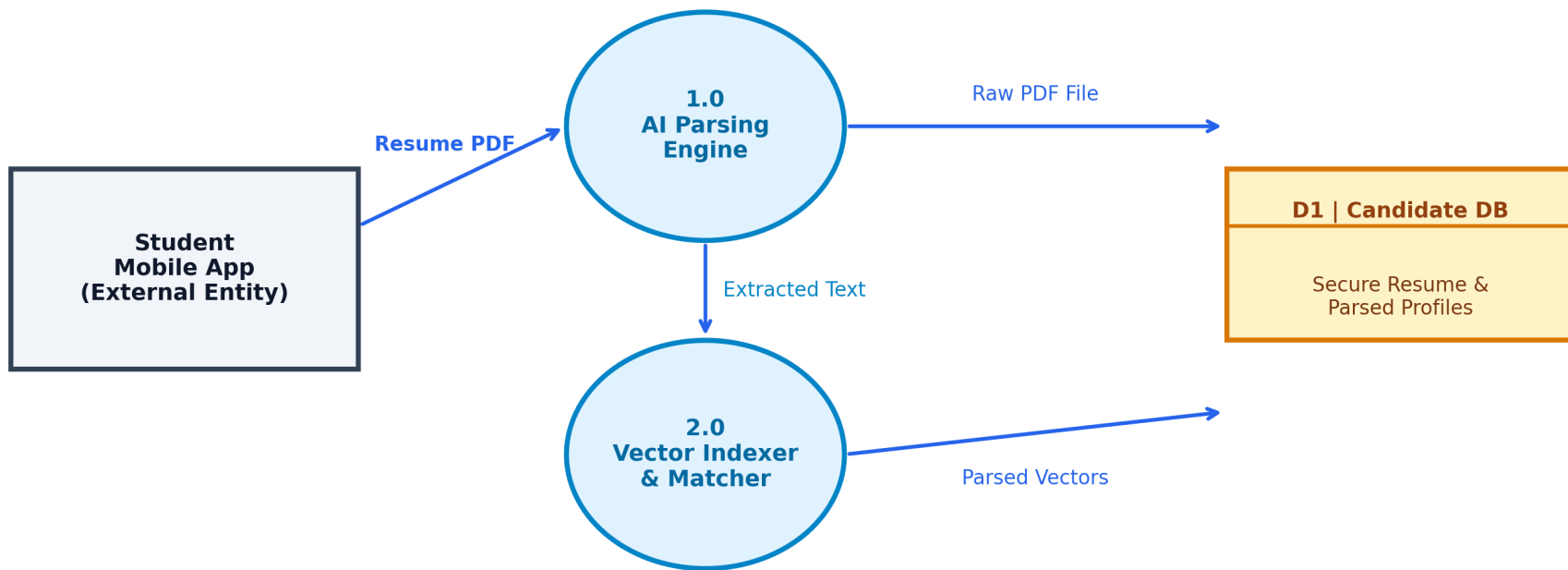
Standard Notations & Visual Grammar Matrix:

Symbol / Notation	Visual Feature	Architectural Semantics
External Entity (Rect)	Source or sink of data outside system boundaries.	Student Mobile App.
Process (Circle)	Transforms incoming data into outgoing data.	1.0 AI Parsing Engine, 2.0 Vector Indexer.
Data Store (Open Rect)	A repository where data is held for later use.	D1 Candidate DB.
Data Flow (Arrow)	Named directed arrows representing data packet movement.	Resume PDF, Extracted Text, Parsed Vectors.

Key Design Insight: Label every arrow with the exact data payload (e.g., 'Resume PDF') and ensure processes use numbered prefixes (1.0, 2.0).

4. Data Flow Diagram (DFD Level 1) — High-Res Precision Visual Model

Data Flow Diagram (DFD Level 1): Resume PDF Upload & AI Processing



Key Design Insight: Diagram Model Execution Note: This model illustrates the 'Tracing a user's resume PDF file from mobile upload, through the AI parsing engine and vector matcher, into the secure database.' scenario. All shapes, line anchors, and decision labels are rendered with precise spatial alignment.

5. Wireframe / Low-Fidelity UI Mockup

Core Architectural Purpose: A wireframe provides a skeletal outline of a visual user interface, establishing spatial layout, content hierarchy, and interactive elements before graphic design.

Detailed Scenario Walkthrough: Designing a mobile career app home screen layout with a search bar, banner carousel, feature grid, and bottom navigation bar.

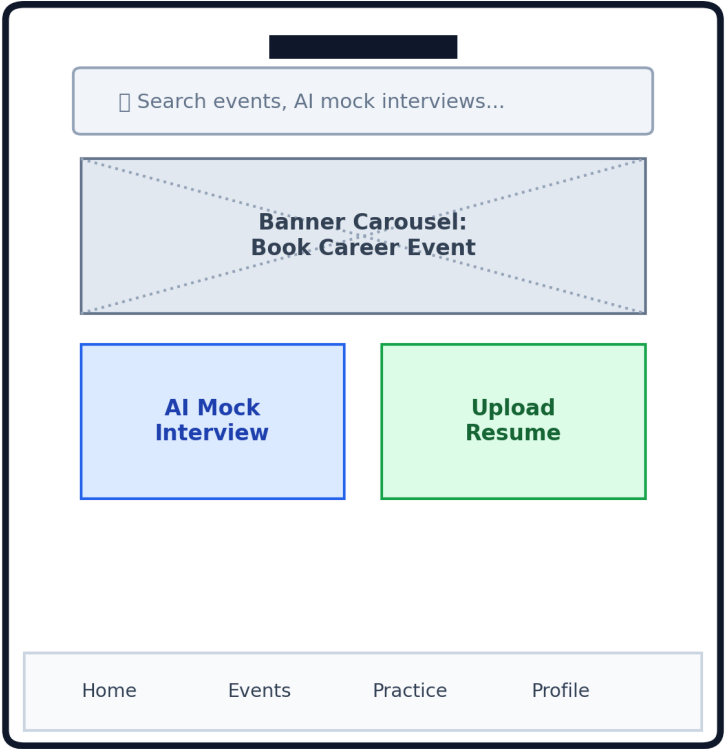
Standard Notations & Visual Grammar Matrix:

Symbol / Notation	Visual Feature	Architectural Semantics
Device Frame	Establishes viewport bounds and aspect ratio.	Smartphone viewport with camera notch.
Placeholder Box	Reserves screen space for dynamic content.	Banner Carousel with diagonal X pattern.
Action Card	Interactive touch points for primary user goals.	AI Mock Interview & Upload Resume cards.
Navigation Bar	Persistent bottom menu bar for core navigation.	Home, Events, Practice, Profile tabs.

Key Design Insight: Use subtle greyscale tones (#F1F5F9 to #334155) so the visual focus remains on structural layout rather than visual embellishment.

5. Wireframe / Low-Fidelity UI Mockup — High-Res Precision Visual Model

Wireframe / Low-Fidelity UI Mockup: Student Career Mobile App



Key Design Insight: Diagram Model Execution Note: This model illustrates the 'Designing a mobile career app home screen layout with a search bar, banner carousel, feature grid, and bottom navigation bar.' scenario. All shapes, line anchors, and decision labels are rendered with precise spatial alignment.

6. Swimlane Diagram (Cross-Functional Flowchart)

Core Architectural Purpose: A Swimlane diagram organizes a multi-step process into parallel horizontal or vertical lanes to explicitly map functional responsibilities across departments.

Detailed Scenario Walkthrough: Tracking a software defect ticket across three distinct organizational lanes: QA Tester -> Developer -> Release Manager.

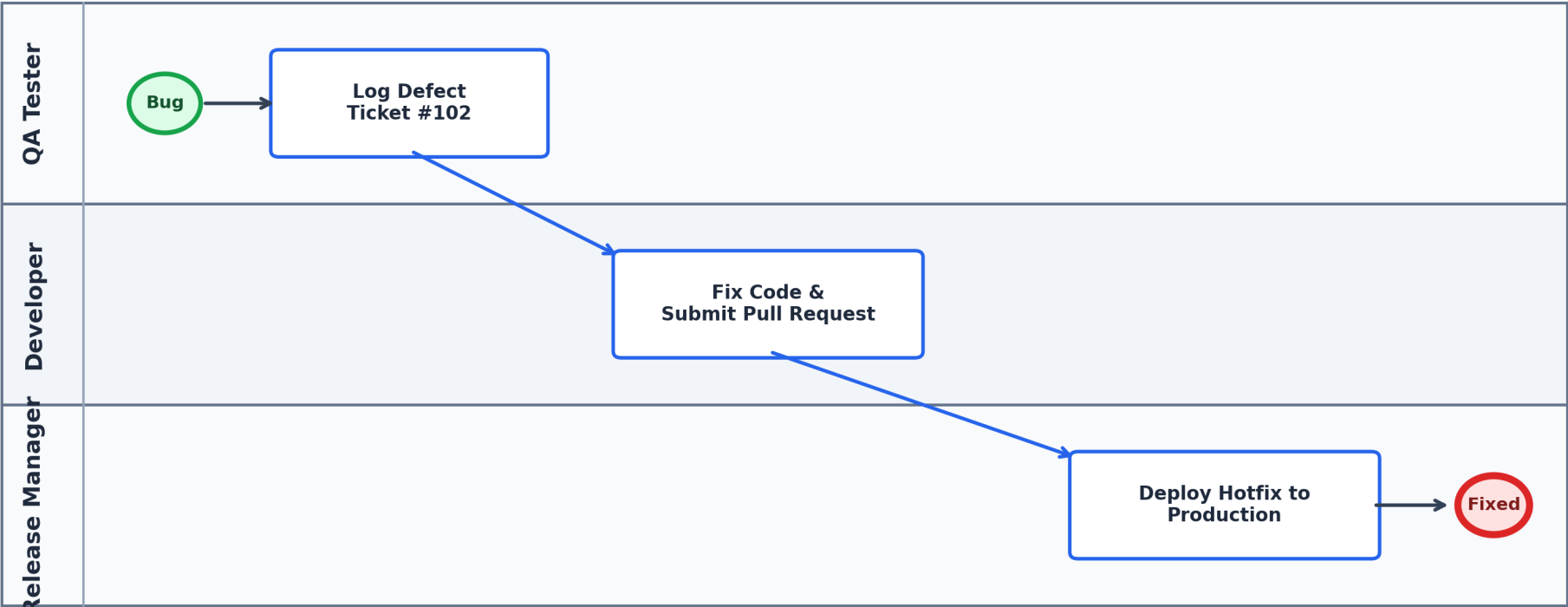
Standard Notations & Visual Grammar Matrix:

Symbol / Notation	Visual Feature	Architectural Semantics
Swimlane Divider	Separates teams, departments, or software roles.	QA Tester, Developer, Release Manager lanes.
Hand-off Arrow	Sequence flow crossing lane boundaries.	QA logs bug -> Dev fixes code -> RM deploys fix.
Start / End Event	Marks process entry and final production state.	Bug reported start -> Hotfix deployed end.
Action Step	Specific task executed within a single lane.	Fix Code & Submit Pull Request.

Key Design Insight: Keep inter-lane arrows strictly perpendicular or smooth diagonal vectors to eliminate line overlapping and confusion.

6. Swimlane Diagram (Cross-Functional Flowchart) — High-Res Precision Visual Model

Swimlane Diagram (Cross-Functional): Software Defect Resolution Flow



Key Design Insight: Diagram Model Execution Note: This model illustrates the 'Tracking a software defect ticket across three distinct organizational lanes: QA Tester -> Developer -> Release Manager.' scenario. All shapes, line anchors, and decision labels are rendered with precise spatial alignment.

7. Gantt Chart

Core Architectural Purpose: A Gantt chart is a horizontal bar chart that illustrates project schedules, task start/end dates, milestones, and work breakdown dependencies.

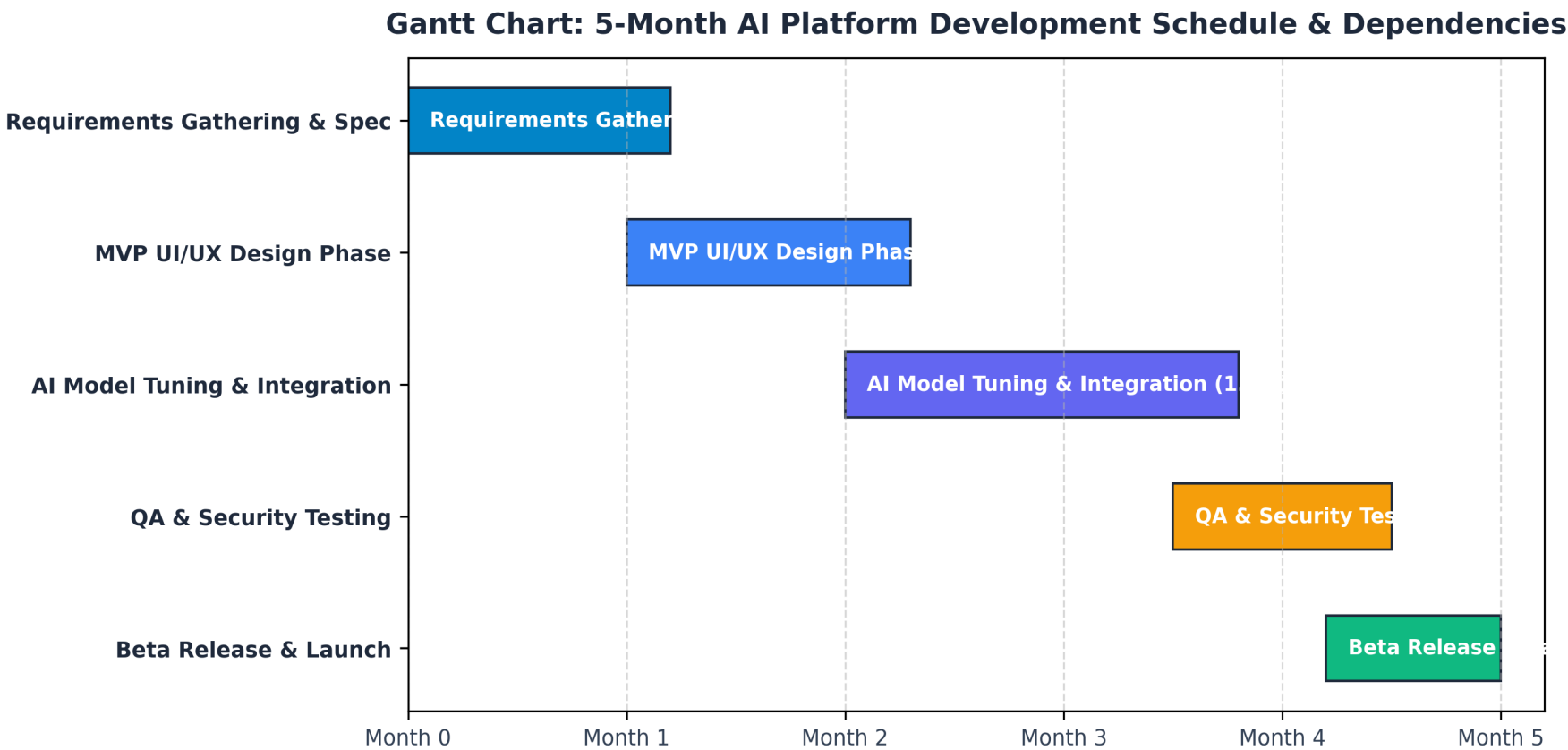
Detailed Scenario Walkthrough: Plotting a 5-month project timeline covering Requirements, MVP Design, AI Model Tuning, QA Testing, and Beta Release.

Standard Notations & Visual Grammar Matrix:

Symbol / Notation	Visual Feature	Architectural Semantics
Task Bar	Horizontal bar representing duration of a task.	Requirements (1.2m), MVP Design (1.3m).
Time Axis (X-axis)	Units of time establishing project schedule.	Month 0 through Month 5.
Task Name (Y-axis)	Structured task list ordered by start date.	Ordered from initial spec to final launch.
Progress Color	Visual coding for distinct project phases.	Blue for Design, Purple for AI, Green for Launch.

Key Design Insight: Ensure task bars have crisp borders and distinct phase colors, with time gridlines subtly visible in the background.

7. Gantt Chart — High-Res Precision Visual Model



Key Design Insight: Diagram Model Execution Note: This model illustrates the 'Plotting a 5-month project timeline covering Requirements, MVP Design, AI Model Tuning, QA Testing, and Beta Release.' scenario. All shapes, line anchors, and decision labels are rendered with precise spatial alignment.

8. Stakeholder Map (Power vs. Interest Grid)

Core Architectural Purpose: A Stakeholder Map evaluates project stakeholders based on their level of power (influence) and interest, guiding engagement and communication strategies.

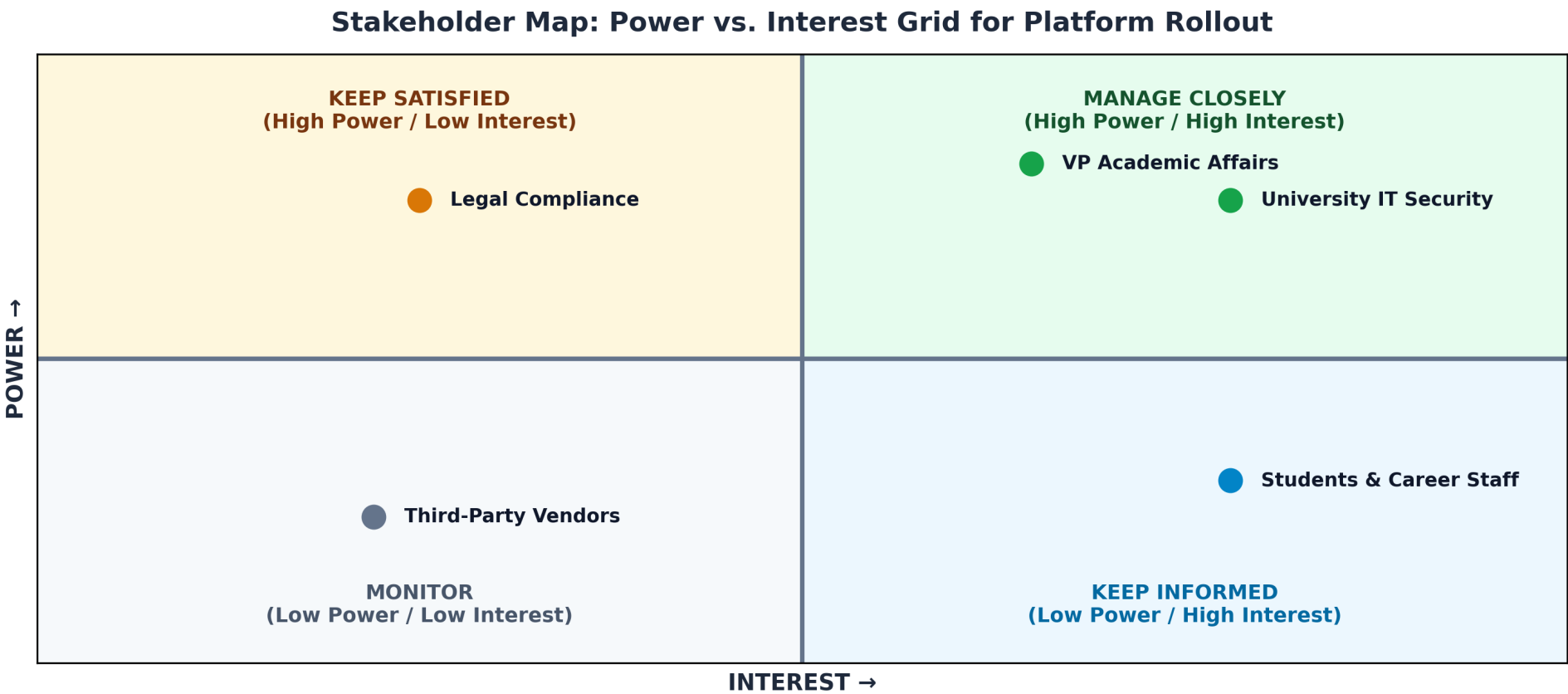
Detailed Scenario Walkthrough: Categorizing university stakeholders for an AI career platform rollout (IT Security, VP Academic Affairs, Legal, Students).

Standard Notations & Visual Grammar Matrix:

Symbol / Notation	Visual Feature	Architectural Semantics
High Power / High Interest	Manage Closely: Primary decision makers requiring direct engagement.	University IT Security, VP Academic Affairs.
High Power / Low Interest	Keep Satisfied: Key influencers requiring risk management.	Legal & Compliance Department.
Low Power / High Interest	Keep Informed: Passionate end users requiring regular updates.	Students & Career Services Staff.
Low Power / Low Interest	Monitor: Minimal engagement required.	Third-Party Vendors.

Key Design Insight: Use distinct pastel background fills for each quadrant and position stakeholder badges with zero text overlap.

8. Stakeholder Map (Power vs. Interest Grid) — High-Res Precision Visual Model



Key Design Insight: Diagram Model Execution Note: This model illustrates the 'Categorizing university stakeholders for an AI career platform rollout (IT Security, VP Academic Affairs, Legal, Students).' scenario. All shapes, line anchors, and decision labels are rendered with precise spatial alignment.

9. UML Statechart Diagram

Core Architectural Purpose: A Statechart diagram models the dynamic behavior of an entity by illustrating its discrete states and event-driven transitions throughout its lifecycle.

Detailed Scenario Walkthrough: Modeling the lifecycle of a Job Application status from Draft -> Submitted -> Under Review -> Accepted / Rejected.

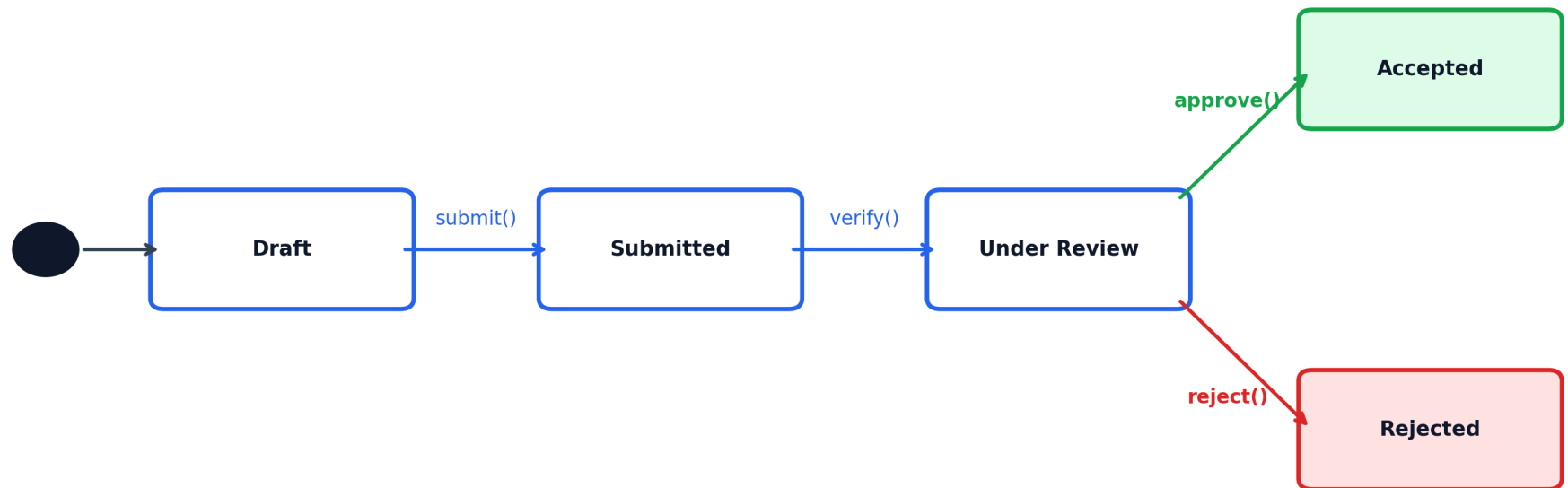
Standard Notations & Visual Grammar Matrix:

Symbol / Notation	Visual Feature	Architectural Semantics
Initial State (Solid Dot)	The starting entry point of an object lifecycle.	Solid black circle initiating Draft state.
State (Rounded Rect)	A condition or situation in which an object exists.	Draft, Submitted, Under Review, Accepted, Rejected.
Transition (Arrow)	Event-triggered movement from one state to another.	submit(), verify(), approve(), reject().
Final State (Bullseye)	The final termination state of the object lifecycle.	Double circle with solid inner dot.

Key Design Insight: Explicitly label transition arrows with trigger event signatures (e.g., approve()) and color-code terminal success/failure states.

9. UML Statechart Diagram — High-Res Precision Visual Model

UML Statechart Diagram: Job Application Lifecycle Transitions



Key Design Insight: Diagram Model Execution Note: This model illustrates the 'Modeling the lifecycle of a Job Application status from Draft -> Submitted -> Under Review -> Accepted / Rejected.' scenario. All shapes, line anchors, and decision labels are rendered with precise spatial alignment.

10. Requirement Traceability Matrix (RTM Grid)

Core Architectural Purpose: An RTM is a structured grid linking product requirements from business origin to technical implementation, test cases, and bug tickets.

Detailed Scenario Walkthrough: Linking REQ-1.1 (Push Notifications) to Mobile App Core, Test Case TC-045, and Defect Ticket BUG-102.

Standard Notations & Visual Grammar Matrix:

Symbol / Notation	Visual Feature	Architectural Semantics
Requirement ID	Unique identifier tracing business requirements.	REQ-1.1, REQ-1.2, REQ-1.3, REQ-1.4.
System Module	Technical component satisfying the requirement.	Mobile App Core, Backend AI Vectorizer.
Test Case ID	Verification suite validating compliance.	TC-045, TC-048, TC-012, TC-089.
Defect / Status	Real-time delivery status and active bug tracking.	PASSED, BUG-102 (In Fix), BUG-114 (Open).

Key Design Insight: Format table headers with dark fills and use color-coded status highlights (Green for PASSED, Red for BUG) for rapid assessment.

10. Requirement Traceability Matrix (RTM Grid) — High-Res Precision Visual Model

Requirement Traceability Matrix (RTM): End-to-End Delivery Verification

Req ID	Requirement Description	System Module	Test Case ID	Defect Ticket / Status
REQ-1.1	Push Notification Alerts	Mobile App Core	TC-045	BUG-102 (In Fix)
REQ-1.2	AI Resume Parsing Engine	Backend AI Vectorizer	TC-048	PASSED
REQ-1.3	SSO Campus Authentication	Identity Security	TC-012	PASSED
REQ-1.4	Mock Interview Audio Stream	Realtime Audio Engine	TC-089	BUG-114 (Open)

Key Design Insight: Diagram Model Execution Note: This model illustrates the 'Linking REQ-1.1 (Push Notifications) to Mobile App Core, Test Case TC-045, and Defect Ticket BUG-102.' scenario. All shapes, line anchors, and decision labels are rendered with precise spatial alignment.

11. Gap Analysis Diagram

Core Architectural Purpose: A Gap Analysis diagram compares an organization's current operational state against its target future state to identify missing capabilities.

Detailed Scenario Walkthrough: Evaluating Current State (manual practice) vs. Future State (24/7 AI coaching), highlighting the LLM API and audio engine gap.

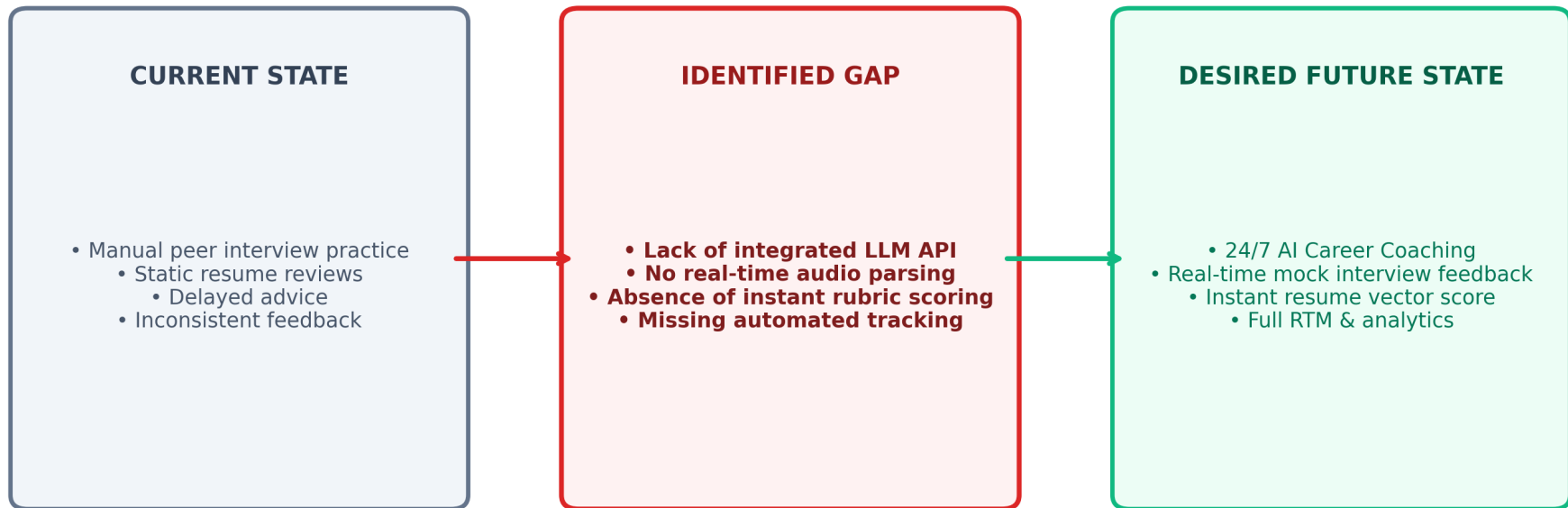
Standard Notations & Visual Grammar Matrix:

Symbol / Notation	Visual Feature	Architectural Semantics
Current State (Pillar 1)	Existing operational baseline and limitations.	Manual peer practice & static resume reviews.
Identified Gap (Bridge)	Missing strategies, APIs, or infrastructure.	Lack of LLM API & real-time audio parsing.
Future State (Pillar 3)	Desired target vision and performance goals.	24/7 AI Coaching & instant vector scoring.
Transition Vector	Directional arrow bridging the capability gap.	Red warning arrow entering Gap; Green entering Future.

Key Design Insight: Emphasize the central Gap box with warning borders (#DC2626) to focus architectural attention on missing enablers.

11. Gap Analysis Diagram — High-Res Precision Visual Model

Gap Analysis Diagram: Transitioning to 24/7 AI Career Coaching Platform



Key Design Insight: Diagram Model Execution Note: This model illustrates the 'Evaluating Current State (manual practice) vs. Future State (24/7 AI coaching), highlighting the LLM API and audio engine gap.' scenario. All shapes, line anchors, and decision labels are rendered with precise spatial alignment.

12. UML Activity Diagram

Core Architectural Purpose: An Activity Diagram models sequential and concurrent workflows of system algorithms or operational processes.

Detailed Scenario Walkthrough: Modeling algorithmic logic deciding whether to display an 'Update Resume' banner based on last login date (> 90 days).

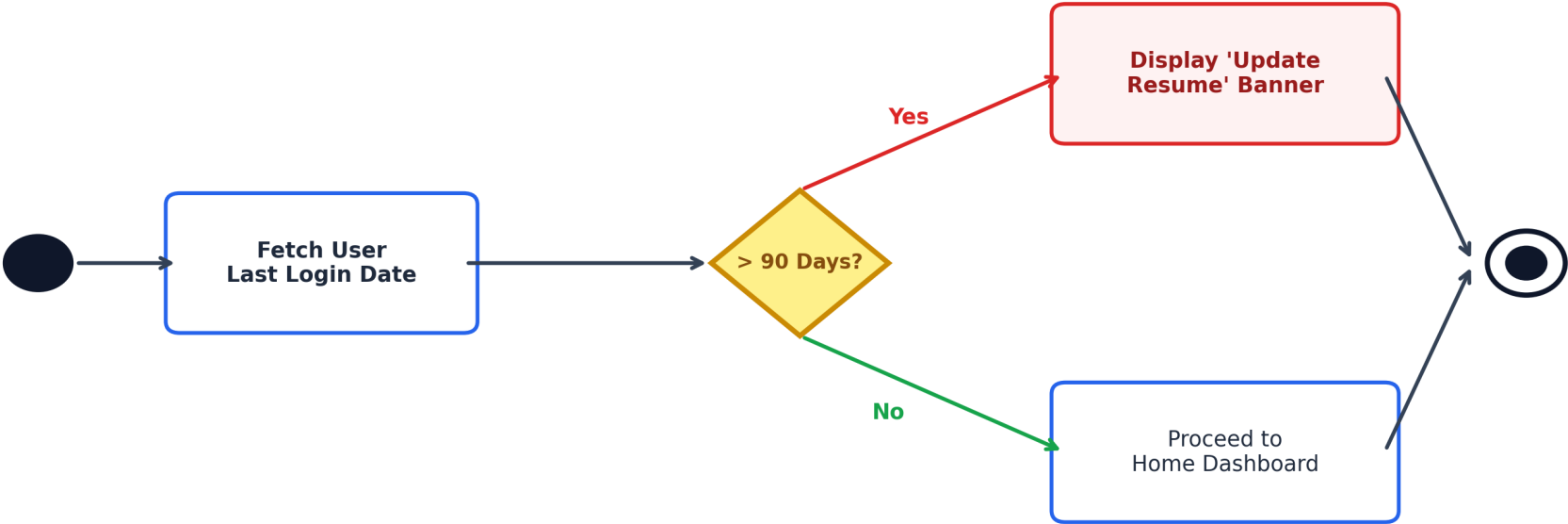
Standard Notations & Visual Grammar Matrix:

Symbol / Notation	Visual Feature	Architectural Semantics
Initial Node	Solid black circle marking workflow start.	Initial node before login check.
Action State	Step where system performs work.	Fetch Last Login Date, Display Banner.
Decision Diamond	Conditional branch evaluating a boolean logic expression.	Days > 90? decision diamond.
Flow Arrows	Directed edges guiding execution path.	Yes branch -> Display Banner; No -> Home Dashboard.

Key Design Insight: Clear directional arrows for decision branches with 'Yes' and 'No' labels positioned right next to decision points.

12. UML Activity Diagram — High-Res Precision Visual Model

UML Activity Diagram: Automated Resume Update Prompt Logic Flow



Key Design Insight: Diagram Model Execution Note: This model illustrates the 'Modeling algorithmic logic deciding whether to display an 'Update Resume' banner based on last login date (> 90 days).' scenario. All shapes, line anchors, and decision labels are rendered with precise spatial alignment.